XLXtrFun Extra Functions for Microsoft® Excel Contents

Zpět na seznam 2D functions

To learn how to use Help, press F1.

A collection of functions which extends the capabilities of Microsoft® Excel; developed primarily to facilitate interpolation of 2-dimensional and 3-dimensional data, and simplify 2-variable curve fitting. XlXtrFun has been used for years by engineering and research and development personnel in Europe, South America, and North America who need to interpolate, extrapolate, and curve fit data rapidly, reliably, and with a virtually non-existent learning curve.

List of Worksheet Functions

List and brief description of all the functions

List of Exported Functions

List and brief description of all the functions

Notes on Error Messages

List and brief description of all the functions

Register your copy (send us your e-mail address) and send suggestions to jdavita@netrax.net. If you register, we will e-mail you updates to the current version (about once a month). We're only talking three working files here, so this will not require any lengthy re-install, just overwrite the old files with the new ones.

Right now (Fall/Winter 1998/99), we are finishing up the help files. Our next project is three-dimensional interpolation with scattered data instead of data with constant X's as is required now.

Χαρισ και ειρηνη

Sar

List of Functions

1D Numerical Functions

LookupClosestValue
Returns the element in an array which is closest to a given value

IndexOfClosestValue
Returns the 1-based index of the element in an array which is closest to a given value

2D Numerical Functions

LookupClosestValue2D
Returns the element in a 2-D array which is closest to two given values

Pfit
Returns the Y which lies on the polynomial line fit by the least squares method

PfitData
Returns the coefficients and statistics for the polynomial line fit by the least squares method

Spline
Returns the Y which lies on the cubic (or natural) spline curve at the given X

Interpolate
Returns the Y which lies on an interpolated curve at the given X

Interp
Returns the Y which lies on an interpolated curve at the given X using the defaults of Interpolate

XatY
Returns the X value at the Max. (Peak),. Min (Valley), or Given Y of an interpolated curve

Intersect
Returns the X value at the intersection of two lines defined by arrays of points

ddydx
Returns the second derivative of the interpolated curve at the given X using the defaults of Interpolate

dydx
Returns the first derivative of the interpolated curve at the given X using the defaults of Interpolate

3D Numerical Functions

Interp3D
Returns an interpolated Z for known X's, known Y's, known Z's, and a given X and Y using the defaults of Interpolate

InterpMatrix
Given an X, Y, Z matrix like the VH efficiency matrix, interpolates or extrapolates a Z using the defaults of Interpolate

Low Level I/O Functions

InputByte
Inputs a byte from a given I/O port

OutputByte
Outputs a byte to a given I/O port

LookupClosestValue

Returns the element in an array which is closest to a given value.

Syntax

LookupClosestValue(Array , ValueToSeek)

Array is a one-dimensional contiguous group of cells containing numbers, arranged either in a row or a column.

ValueToSeek is a number. LookupClosestValue searches through all the values in *Array* and returns the value in *Array* which is closest to *ValueToSeek*.

Remarks

- LookupClosestValue returns the first value it finds for which there is no closer value in *Array*. LookupClosestValue begins its search at the "first" element in *Array*, which is the upper or left element.

Examples

`LookupClosestValue( {10,20,30,40,50}, 32 )` equals 30

`LookupClosestValue( {10,20,30,40,50}, 35 )` equals 30 because 30 and 40 are both equally close to 35. LookupClosestValue returned 30 because it found 30 first, and no other element was *closer* than 30.

See Also

IndexOfClosestValue, LookupClosestValue2D

IndexOfClosestValue

Returns the index of the element in an array which is closest to a given value. The first element of the array is at index = 1.

Syntax

IndexOfClosestValue(Array , ValueToSeek)

Array is a one-dimensional contiguous group of cells containing numbers, arranged either in a row or a column.

ValueToSeek is a number. IndexOfClosestValue searches through all the values in *Array* and returns index of the value in *Array* which is closest to *ValueToSeek*.

Remarks

- IndexOfClosestValue returns the index for first value it finds for which there is no closer value in *Array*. IndexOfClosestValue begins its search at the "first" element in *Array*.
- The "first" element in *Array* is the upper or left element, and is at index = 1.

Examples

`IndexOfClosestValue( {10,20,30,40,50}, 32 )` equals 3

`IndexOfClosestValue( {10,20,30,40,50}, 35 )` equals 3 because 30 and 40 are both equally close to 35, and IndexOfClosestValue found the third element, 30, first, and no other element was *closer* than 30.

See Also

LookupClosestValue, LookupClosestValue2D

LookupClosestValue2D

Returns the element in a two-dimensional array which is closest to two given values.

Syntax

LookupClosestValue2D(XYArray , ArrayofXKeys , ArrayofYKeys , XValuetoSeek , YValuetoSeek)

XYArray is a two-dimensional contiguous group of cells containing numbers.

ArrayOfXKeys is a one-dimensional contiguous group of cells containing numbers, arranged in a row.

ArrayOfYKeys is a one-dimensional contiguous group of cells containing numbers, arranged in a column.

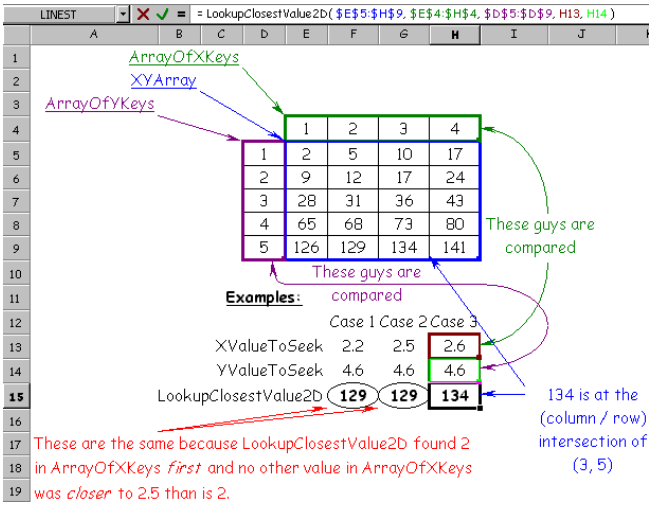
XValueToSeek and YValueToSeek are numbers. LookupClosestValue2D searches through all the values in *ArrayOfXKeys* and *ArrayOfYKeys* and returns the value in *XYArray* which is at the intersection of the column which contains the closest X-Value and the row which contains the closest Y-Value.

Remarks

- LookupClosestValue2D compares each value in *ArrayOfXKeys* to *XValueToSeek*. It settles on the first value it finds for which there is no closer value in the array. LookupClosestValue2D begins its search at the "first" element in the array, which is the left-most element.
- The same is true for *ArrayOfYKeys* to *YValueToSeek*, except that the "first" element in the array is the top-most element.

Examples

{bmct bm0.BMP}



See Also

IndexOfClosestValue, LookupClosestValue

PFit

Returns the Y at a given X on the polynomial curve of a given order least squares fit of data points in two dimensions. Returns the same values as are displayed when Microsoft® Excel inserts a polynomial trendline of the data.

Syntax

PFit(ArrayOfXs , ArrayOfYs, GivenX , Order , *Extrapolate?*)

ArrayOfXs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The independent variable.

ArrayOfYs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column corresponding to *ArrayOfXs*, arranged either in a row or a column. The dependent variable.

GivenX is a number. PFit returns the Y at this X which is on the polynomial fit curve.

Order is an integer indicating the order of the polynomial used to fit the curve to the data.

Extrapolate? Optional. Either TRUE of FALSE; whether to allow extrapolation. Default is FALSE.

Remarks

- For the parameter *Extrapolate?*, 0 (zero) is equivalent to FALSE, and any non-zero number is equivalent to TRUE.
- PFit can be used to return Y values calculated by the polynomial trendline.
- You can use PFit to return values easily, and PFitData to return the coefficients and statistics.
- PFit does not have the flexibility of TREND, but is very easy to use because it does not require the elaborate and hard-to-change set up of TREND.

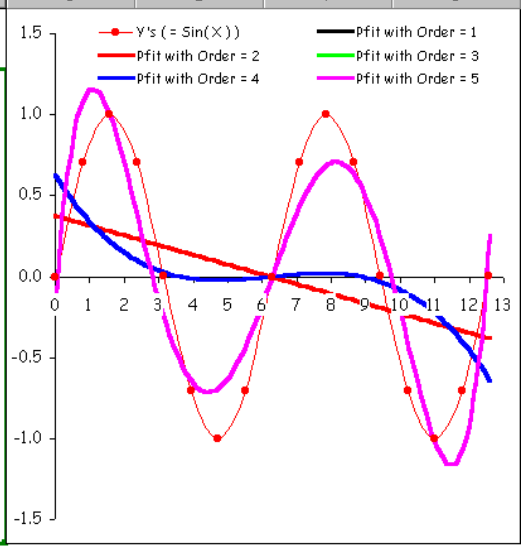
Examples

- In the examples below, an array of x's was (arbitrarily) created from $0 \le x \le 4\pi$, in increments of $\pi/4$. An array of y's was (arbitrarily) created from the equation y = Sin(x).
- Five different PFit orders are shown, *Order* = 1, 2, 3, 4, and 5. By entering different values in the yellow cells, the order of fit is changed, and the closeness of fit can be checked.
- It is interesting in this particular example that in fitting data to the sine curve, *Order* = 1 is the same as *Order* = 2; and *Order* = 3 is the same as *Order* = 4.

{bmct bm1.BMP}

LINEST = PFit($B$2:$B$18, $C$2:$C$18, $B22, C$20)

	A	B	C	D	E	F	G
1	Input Data:	X's	Y's (= Sin(X))				
2		0.00	0.0000				
3		0.79	0.7071				
4		1.57	1.0000				
5		2.36	0.7071				
6		3.14	0.0000				
7		3.93	-0.7071				
8		4.71	-1.0000				
9		5.50	-0.7071				
10		6.28	0.0000				
11		7.07	0.7071				
12		7.85	1.0000				
13		8.64	0.7071				
14		9.42	0.0000				
15		10.21	-0.7071				
16		11.00	-1.0000				
17		11.78	-0.7071				
18		12.57	0.0000				
20	Order of Fit:		1	2	3	4	5
21		Given X's	Pfit with Order = 1	Pfit with Order = 2	Pfit with Order = 3	Pfit with Order = 4	Pfit with Order = 5
22		0	0.3787	0.3787	0.6288	0.6288	-0.1586
23		0.1	0.3727	0.3727	0.5951	0.5951	0.1072
24		0.2	0.3666	0.3666	0.5626	0.5626	0.3370
25		0.3	0.3606	0.3606	0.5312	0.5312	0.5332



See Also

PFitData, TREND, Interpolate, Interp, Spline, Interp3D, InterpMatrix

PFitData

Returns the coefficients for the polynomial curve of a given order least squares fit of data points in two dimensions. Returns the same values as Microsoft® Excel LINEST function except that the coefficients are in forward order and zeros are returned for all values not applicable. This allow the user to easily set up and test various orders of fit.

Syntax

PFitData(ArrayOfXs , ArrayOfYs, Order , *False=>(0,0)*)

ArrayOfXs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The independent variable.

ArrayOfYs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column corresponding to *ArrayOfXs*, arranged either in a row or a column. The dependent variable.

Order is an integer indicating the order of the polynomial used to fit the curve to the data.

False=>(0,0) Optional. Either TRUE of FALSE; False forces the curve to go through (0,0), True means the Y-Intercept is calculated normally. Same as LINEST *Const*.

Remarks

- Internally. PFitData calls the Microsoft® Excel LINEST function. Therefore, it has the same limitations as LINEST, the main one being the maximum order of fit = 16.
- PFitData always returns an array 5 rows by 17 columns. You do not have to enter PFitData as an array formula, but for large workbooks, you can achieve significant performance improvement if you take advantage of entering it as an array formula. For more information about array formulas, click Array Formulas
- Although the 5 x 17 array is always returned, you do not have to display all of the elements. For example, filling a 3 x 5 range of cells with the array formula
  `= PFitData( {0.57,0.41,0.59,1.02,1.3,1.2,0.9}, {0,0.1,0.2,0.3,0.4,0.5,0.6}, 3 )`
  yields,

0.5615	-2.925	5.1148	-2.281	0	0
1.9664	7.7604	9.4746	3.6395	0	0
0.6753	0.1741	0	0	0	0

- The first two rows of the returned array are the same as LINEST, except that the coefficients are returned in the opposite order as returned by LINEST. This is a more intuitively sensible arrangement because the exponent for X increases as you go to the right. This also results in the coefficient for X^4, for example, always staying in the same cell regardless of the order of fit.
- For any non-applicable exponent, PFitData returns zero for the coefficient. For example, if you specify *Order* = 4, the coefficients for X^5, X^6, X^7, …, X^16 returned by PFitData will be zero.
- For the preceding two reasons, you do not need elaborate formulas which shift coefficients around in order to match coefficients and exponents depending on *Order*.
- For the parameter *False=>(0,0)*, 0 (zero) is equivalent to FALSE, and any non-zero number is equivalent to TRUE.
- PFitData does not have the flexibility of LINEST, but is very easy to use because it does not require the elaborate and hard-to-change set up of LINEST.
- As with LINEST, you can use the INDEX function to return any one element of the returned array. For example, the formula,
  `= INDEX( PFitData( {0.57,0.41,0.59,1.02,1.3,1.2,0.9}, {-0,0.1,0.2,0.3,0.4,0.5,0.6}, 3 ), 3, 1 )`

returns `0.6753`, the element in row 3, column 1, which happens to be $r_{¿}$.

- You can use PFit to return values easily, and PFitData to return the coefficients and statistics.

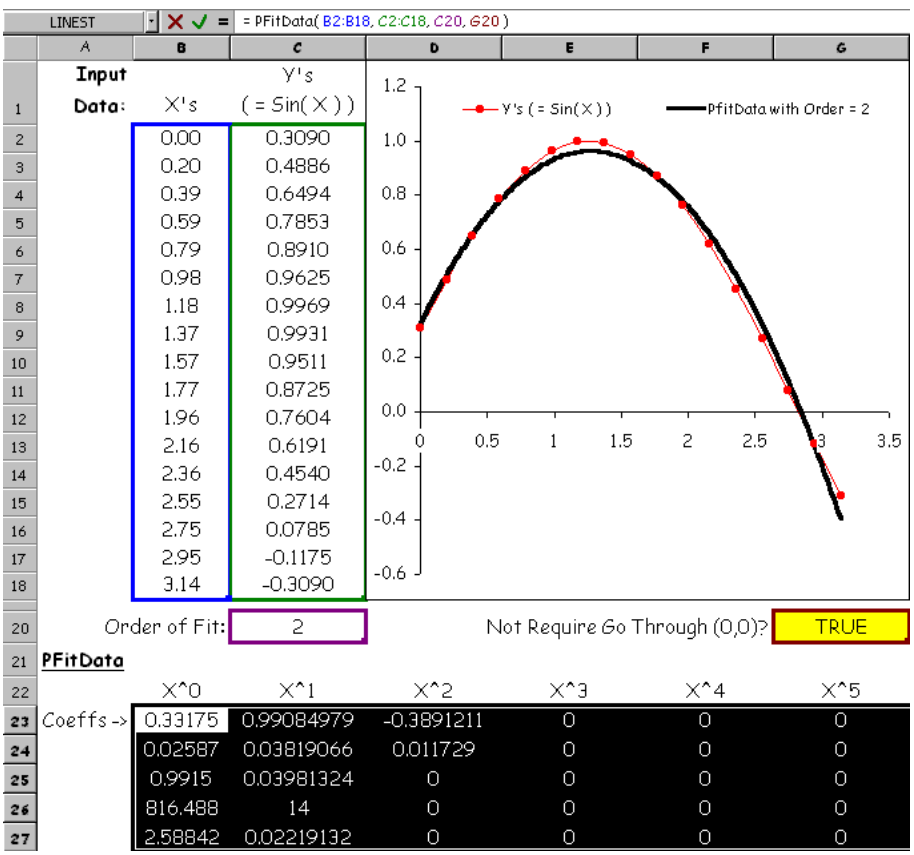
Errors

- If there was an error in the data entered, and if you enter PFitData as an array formula, PFitData will return text describing the problem: “Order < 0", “False=>(0,0) doesn't make sense”, or “Numbers of points don't match".
- If there was an error in the data entered, and you would have received one of the errors above, but you did NOT enter PFitData as an array formula, but rather used the INDEX function to get the individual elements, you will get a #VALUE! error. This is not particularly enlightening since this is the same error Excel gives when it cannot convert the data to the correct type required by the function. See #VALUE!. This is another good reason to enter PFitData as an array formula.
- If you get the error, “LINEST returned xxx“. You will also get data describing what LINEST was given to work with.
- If you get the error, “Excel4 xlfLinest returned error #xxx“. The program could not even call the LINEST function. It should be impossible to get this error. (I’ve never seen it; tell me if you do!)

Examples

- In the example below, an array of x's was (arbitrarily) created from $0 \le x \le \pi$, in increments of $\pi/16$. An array of y's was (arbitrarily) created from the equation $y = \mathrm{Sin}(x + \pi / 10)$.
- The array of coefficients is in the first row.

{bmct bm2.BMP}



See Also

PFit, LINEST ,Interpolate, Interp, Spline, Interp3D, InterpMatrix

Spline Zpět na seznam 2D functions

Returns the Y at a given X on the natural cubic spline curve defined by a given set of points.

Syntax

Spline(ArrayOfXs , ArrayOfYs, GivenX , *Extrapolate?*)

ArrayOfXs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The values must be either constantly increasing or constantly decreasing.

ArrayOfYs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. These values are the Y's corresponding to the *ArrayOfXs* above.

GivenX is a number. Spline returns the Y at this X which is on the natural cubic spline curve.

Extrapolate? Optional. Either TRUE of FALSE; whether to allow extrapolation. Default is FALSE.

Remarks

- For the parameter *Extrapolate?*, 0 (zero) is equivalent to FALSE, and any non-zero number is equivalent to TRUE.
- The points in *ArrayOfXs* and *ArrayOfYs* are called the **knots**.
- The natural cubic spline is a third degree piecewise polynomial curve that 1) goes through all the knots, 2) at each knot, the first derivatives of the two curves that meet there are equal, 3) at each knot, the second derivatives of the two curves that meet there are equal, and 3) at the two end knots, the second derivatives of each curve equal zero.
- Both the natural cubic spline and double parabolic interpolation (see Interpolate and Interp) have a continuous first derivative (slope). However, only the natural cubic spline has a continuous second derivative (acceleration).
- Because the natural cubic spline has a continuous second derivative, all of the knots are required to define all the polynomials that make up the entire curve, Thus, changing any one knot changes all the interpolated values that are between knots. Interpolate and Interp do not have this characteristic because they do not have a continuous second derivative. The double parabolic method of Interpolate and Interp considers only the four knots surrounding *GivenX* to define the piecewise interpolated curves.
- As can be seen in the example below, even though there are three collinear knots (actually two sets of three collinear knots), the natural cubic spline never "figures out" that the knots define a straight line, and keeps oscillating up and down between knots. On the other hand, Interpolate and Interp do draw a straight line. Spline also has greater fluctuations between knots. (For these two reasons we developed the double parabolic method of Interpolate and Interp.)
- Spline returns #NUM! if it cannot calculate the cubic spline. There are two frequent causes. 1) Extrapolation is required, and *Extrapolate?* is not set to TRUE. Although it shouldn't be a problem, this sometimes happens when *GivenX* exactly equals the X-value of either of the end knots. In such a case, you will have to set *Extrapolate?* to TRUE. 2) *ArrayOfXs* is not constantly increasing or decreasing. For example, the following two *ArrayOfXs* are illegal:

```
{ 1, 2, 3, 4, 3.5, 5 }
{ 5, 4, 3, 2, 2, 1 }
```

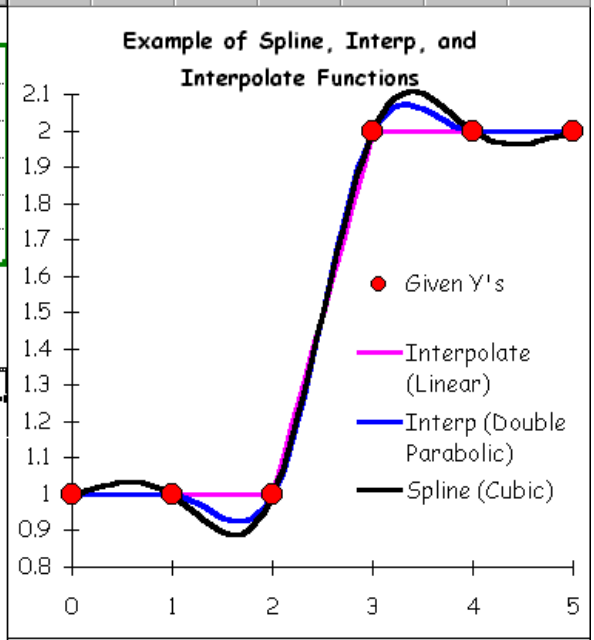
Examples

- In this example, the three different interpolation methods are shown: natural cubic spline, double parabolic interpolation, and linear interpolation.

{bmct bm3.BMP}

LINEST = Spline($C$2:$C$7, $D$2:$D$7, $A9)

	A	B	C	D
1			Given X's	Given Y's
2			0	1
3			1	1
4			2	1
5			3	2
6	51 Points		4	2
7	0.1 = Increment		5	2
8	x	Interpolate (Linear)	Interp (Double Parabolic)	Spline (Cubic)
9	0	1	1	1
10	0.1	1	1	1.009
11	0.2	1	1	1.01745
12	0.3	1	1	1.02482
13	0.4	1	1	1.03055
14	0.5	1	1	1.03409
15	0.6	1	1	1.03491
16	0.7	1	1	1.03245
17	0.8	1	1	1.02618
18	0.9	1	1	1.01555
19	1	1	1	1
20	1.1	1	0.9955	0.97955
21	1.2	1	0.984	0.95636
22	1.3	1	0.9685	0.93318
23	1.4	1	0.952	0.91273
24	1.5	1	0.9375	0.89773



By default, *Interpolate*, *Interp*, *XatY*, *Intersect*, *dydx*, *ddydx*, *Interp3D*, and *InterpMatrix* use a third-order piecewise polynomial for interpolation; and a second-order polynomial for extrapolation. *Spline*, on the other hand, uses a *third*-order polynomial for extrapolation.

See Also

Interpolate, Interp, PFit, Interp3D, InterpMatrix

Interpolate Zpět na seznam 2D functions

Returns the Y at a given X on the interpolated curve defined by a given set of points.

Syntax

Interpolate(ArrayOfXs , ArrayOfYs, GivenX , *Extrapolate?* , *Parabolic?* , *Averaging?* , *SmoothingPower*)

ArrayOfXs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The values must be either constantly increasing or constantly decreasing.

ArrayOfYs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. These values are the Y's corresponding to the *ArrayOfXs* above.

GivenX is a number. Interpolate returns the Y at this X which is on the interpolate curve.

Extrapolate? Optional. Either TRUE of FALSE; whether to allow extrapolation. Default is FALSE.

Parabolic? Optional. Either TRUE of FALSE. If TRUE, Interpolate develops a smooth "double parabolic" curve between the given set of points. If FALSE, linear interpolation is used. Default is TRUE.

Averaging? Optional. Either TRUE of FALSE. If TRUE, and *Parabolic?* = TRUE, the interpolated curve is averaged from both directions for curve smoothing (slope matching). If FALSE, and *Parabolic?* = TRUE, for each interval, a parabola is created using the point to the right and the two points to the left. Default is TRUE.

SmoothingPower Optional. The weight used for averaging is raised to this power. Default is 1.

Remarks

- When you want to use this function and accept the default values for the optional parameters, use the Interp function. Interp uses the same numerical routines as this function but assumes the default values for the optional parameters; i.e., *Extrapolate?* = FALSE, *Parabolic?* = TRUE, *Averaging?* = TRUE, and *SmoothingPower* = 1.
- As an alternative to entering TRUE or FALSE for the parameters *Extrapolate?*, *Parabolic?*, and *Averaging?*, 0 (zero) is equivalent to FALSE, and any non-zero number is equivalent to TRUE.
- The points in *ArrayOfXs* and *ArrayOfYs* are called the **knots**.
- If *Parabolic?* = FALSE, interpolation is linear, and the parameters *Averaging?* and *SmoothingPower* are ignored.
- If *Parabolic?* = TRUE (default), and there are greater than two knots, parabolic interpolation is used. If there are only two knots, only linear interpolation is possible.
- In the following context, "before" and "after", and "left" and "right" refer to the order in which the knots are in given in (*ArrayOfXs*, *ArrayOfYs*), not their numerical value. Their left/right placement on the number line is irrelevant.
- If *Parabolic?* = TRUE (default) and *Averaging?* = FALSE, one parabola is used to define the curve for each interval. If possible, the parabola is defined by the two knots before the interval plus the knot after the interval. Note that the first two intervals use the same three knots, so the curves are the same parabola. The last two intervals have unique curves because they each use a different set of three knots.
- If *Parabolic?* = TRUE (default) and *Averaging?* = TRUE (default), two parabolas are used to define the curve for each interval. One parabola is defined by the two knots before the interval plus the knot after the interval, and a second parabola is defined by the one knot before the interval plus the two knots after the interval. The two calculated Y's are weighted according to how close the given X is to the left or right hand point.

  Weight = ((GivenX - X_of_Knot_on_Left) /
  (X_of_Knot_on_Right - X_of_Knot_on_Left))^*SmoothingPower*

  Returned Y = Y_on_Parabola_Coming_from_the_Right · Weight +

Y_on_Parabola_Coming_from_the_Left · (1 - Weight)

The resultant curve, provided *SmoothingPower* > 0.5, has a continuous first derivative (slope).

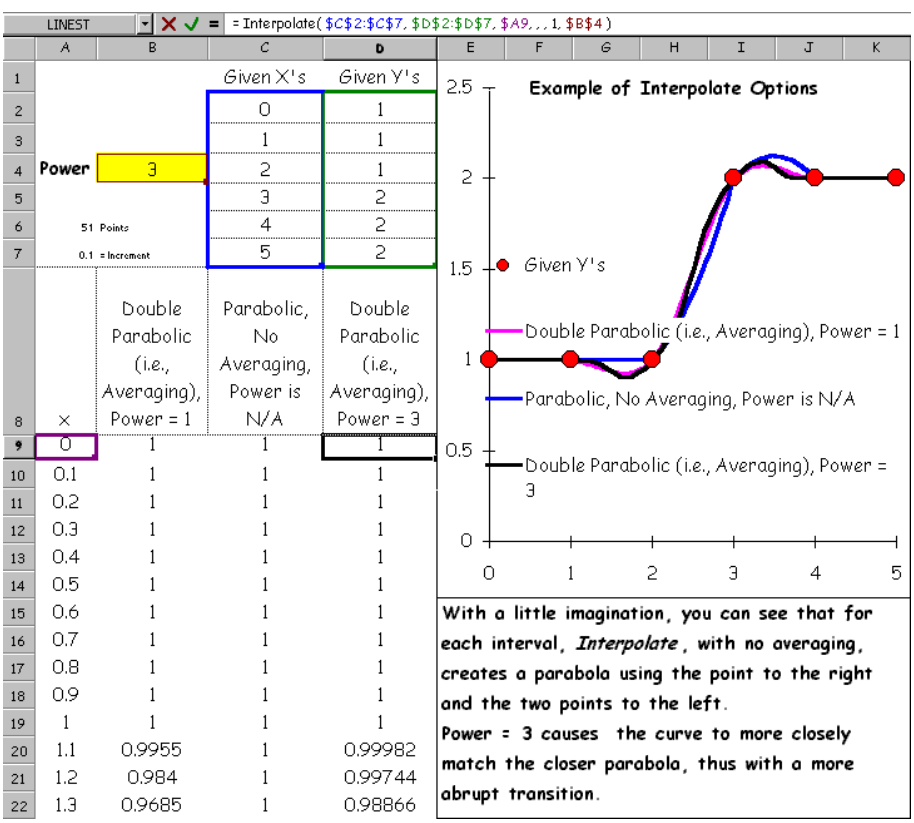
- *SmoothingPower*. For *SmoothingPower* = 1 (default), the weight calculated above is the common inverse distance weighting. The resultant curves between knots are third order polynomials.
- The effect of *SmoothingPower* > 1 is to more closely follow the curve from the nearest given knot for a longer time. If there is an inflection, it will be more abrupt.
- If you are inclined to experiment with different values for *SmoothingPower*, you will find that if you give Interpolate only the peaks and valleys of the Sine curve { (π/2,1) , (3π/2,-1) , (5π/2,1) , (7π/2,-1)…}, an almost perfect fit is obtained using 1.156 for the power.
- See Spline for a detailed discussion of the differences between the curves developed by the Spline and Interpolate functions.
- Interpolate returns #NUM! if it cannot calculate the interpolated Y. There are two frequent causes. 1) Extrapolation is required, and *Extrapolate?* is not set to TRUE. Although it shouldn't be a problem, this sometimes happens when *GivenX* exactly equals the X-value of either of the end knots. In such a case, you will have to set *Extrapolate?* to TRUE. 2) *ArrayOfXs* is not constantly increasing or decreasing. For example, the following two *ArrayOfXs* are illegal:

```
{ 1, 2, 3, 4, 3.5, 5 }
{ 5, 4, 3, 2, 2, 1 }
```

Examples

- In this example, the three different interpolation methods are shown: natural cubic spline, double parabolic interpolation, and linear interpolation.

{bmct bm4.BMP}



See Also

Spline, Interp, PFit, Interp3D, InterpMatrix

Interp

Zpět na seznam 2D functions

Returns the Y at a given X on the interpolated curve defined by a given set of points using the defaults of Interpolate.

Syntax

Interp(ArrayOfXs , ArrayOfYs, GivenX)

ArrayOfXs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The values must be either constantly increasing or constantly decreasing.

ArrayOfYs one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. These values are the Y's corresponding to the *ArrayOfXs* above.

GivenX is a number. Interpolate returns the Y at this X which is on the interpolate curve.

Remarks

- Interp uses the defaults of Interpolate. Therefore, if you need to extrapolate, you must use Interpolate(ArrayOfXs, ArrayOfXs, GivenX, TRUE).

Examples

- See Interpolate.

See Also

Interpolate, Spline, PFit, Interp3D, InterpMatrix

XatY

Zpět na seznam 2D functions

Returns the X value at the maximum (Peak), minimum (Valley), or given Y of an interpolated curve using the defaults of Interpolate. XatY is essentially the inverse of Interpolate.

Syntax

XatY(KnownXArray , KnownYArray , PeakValleyOrY , GuessX , GivenY , Extrapolate?)

KnownXArray one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. The values must be either constantly increasing or constantly decreasing.

KnownYArray one-dimensional array of numbers, or a contiguous group of cells containing numbers arranged either in a row or a column. These values are the Y's corresponding to the *KnownXArray* above.

PeakValleyOrY Optional. Either “P”, “p”, “V”, “v”, “Y”, or “y” indicating whether to return X at the interpolated maximum Y (*PeakValleyOrY* = “P” or “p” for peak), minimum Y (*PeakValleyOrY* = “V” or “v” for valley), or the interpolated X at a given Y (*PeakValleyOrY* = “Y” or “y”). *PeakValleyOrY* must be a text value enclosed in quotation marks, or a reference to a cell containing a text value. Default is “P”.

GuessX Optional. In trying to solve for either the peak, valley, or Y, XatY will start looking with X = *GuessX*. Default *GuessX* = X at the greatest Y if *PeakValleyOrY* = “P” or “p”, X at the least Y if *PeakValleyOrY* = “V” or “v”, or X at the Y closest to *GivenY* if *PeakValleyOrY* = “Y” or “y”.

GivenY Ignored for *PeakValleyOrY* = “Y”, “y”, “V” or “v”; required for *PeakValleyOrY* = “Y” or “y”. When *PeakValleyOrY* = “Y” or “y”, XatY will return the X which yields the interpolated *GivenY*.

Extrapolate? Optional. Either TRUE of FALSE; whether to allow extrapolation. Default is FALSE.

Remarks

- With the exception of *Extrapolate?*, XatY uses the defaults of Interpolate; there is no function (at the present time) which will return the X at a given Y using Spline, PFit, or linear interpolation.

Examples

- See Interpolate.

See Also

Interpolate, Spline, PFit, Interp3D, InterpMatrix

Intersect

Zpět na seznam 2D functions

See Also

Interpolate, Spline, PFit, Interp3D, InterpMatrix

ddydx

Zpět na seznam 2D functions

Returns the second derivative of the interpolated curve at the given X using the defaults of Interpolate.

See Also

dydx, Interpolate, Spline, PFit, Interp3D, InterpMatrix

dydx

Zpět na seznam 2D functions

Returns the first derivative of the interpolated curve at the given X using the defaults of Interpolate.

See Also

ddydx, Interpolate, Spline, PFit, Interp3D, InterpMatrix

Interp3D

{bmct bm5.BMP}

SUM = Interp3D($B$5:$B$68, $C$5:$C$68, $D$5:$D$68, $F5, $G5, , J5, K5)

Interp3DAndInterpMatrix

(Data for InterpMatrix Starts in Cell $F$14)

Data for Interp3d Below

	X's in blocks of constant values	Y's	Calculated Z (=X²+Y²)
5	-1.5	-2	6.25
6	-1.5	-1.429	4.29082
7	-1.5	-0.857	2.98469
11	-1.5	1.4286	4.29082
12	-1.5	2	6.25
13	-1.071	-2	5.14796
14	-1.071	-1.429	3.18878
19	-1.071	1.4286	3.18878
20	-1.071	2	5.14796
21	-0.643	-2	4.41327
22	-0.643	-1.429	2.45408
23	-0.643	-0.857	1.14796
24	-0.643	-0.286	0.4949
25	-0.643	0.2857	0.4949

Blocks of constant X's either constantly increasing or decreasing

Within each X block, Y's are either constantly increasing or decreasing

Examples:

GivenX	GivenY	Calculated Z (=X²+Y²) (What the Interpolated Z Should Equal)	Interp3D Z	Constant X Interpolation Method for Interp3D	Constant Y Interpolation Method for Interp3D	InterpMatrix Z	Parabolic Interpolation for InterpMatrix?
0	0	0	1.041E-17	P	P	-4.857E-17	TRUE
0	0	0	0.0459184	P	L		
0	0	0	0.0816327	L	P		
0	0	0	0.127551	L	L	0.127551	FALSE

InterpMatrix Data

Minimum X	Maximum X	Minimum Y	Maximum Y	Number of X's	Number of Y's		
-1.5	1.5	-2	2	8	8		
6.25	6.25	6.25	6.25	6.25	6.25	6.25	6.25
3.19	3.19	3.19	3.19	3.19	3.19	3.19	3.19
1.15	1.15	1.15	1.15	1.15	1.15	1.15	1.15
0.13	0.13	0.13	0.13	0.13	0.13	0.13	0.13
0.13	0.13	0.13	0.13	0.13	0.13	0.13	0.13
1.15	1.15	1.15	1.15	1.15	1.15	1.15	1.15
3.19	3.19	3.19	3.19	3.19	3.19	3.19	3.19
6.25	6.25	6.25	6.25	6.25	6.25	6.25	6.25

Increasing Y

Increasing X

Value for Z at (MinX, MinY)

See Also

ddydx, Interpolate, Spline, PFit, Interp3D, InterpMatrix

InterpMatrix

{bmct bm6.BMP}

LINEST = InterpMatrix($F$14:$M$22, $F5, $G5, , M5)

Interp3DAndInterpMatrix

(Data for InterpMatrix Starts in Cell $F$14)

Data for Interp3d Below

X's in blocks of constant values	Y's	Calculated Z (=X²+Y²)
-1.5	-2	6.25
-1.5	-1.429	4.29082
-1.5	-0.857	2.98469
-1.5	1.4286	4.29082
-1.5	2	6.25
-1.071	-2	5.14796
-1.071	-1.429	3.18878
-1.071	1.4286	3.18878
-1.071	2	5.14796
-0.643	-2	4.41327
-0.643	-1.429	2.45408
-0.643	-0.857	1.14796
-0.643	-0.286	0.4949
-0.643	0.2857	0.4949

Blocks of constant X's either constantly increasing or decreasing

Within each X block, Y's are either constantly increasing or decreasing

Examples:

GivenX	GivenY	Calculated Z (=X²+Y²) (What the Interpolated Z Should Equal)	Interp3D Z	Constant X Interpolation Method for Interp3D	Constant Y Interpolation Method for Interp3D	InterpMatrix Z	Parabolic Interpolation for InterpMatrix?
0	0	0	1.041E-17	P	P	-4.857E-17	TRUE
0	0	0	0.0459184	P	L		
0	0	0	0.0816327	L	P		
0	0	0	0.127551	L	L	0.127551	FALSE

InterpMatrix Data

Minimum X	Maximum X	Minimum Y	Maximum Y	Number of X's	Number of Y's		
-1.5	1.5	-2	2	8	8		
6.25	6.25	6.25	6.25	6.25	6.25	6.25	6.25
3.19	3.19	3.19	3.19	3.19	3.19	3.19	3.19
1.15	1.15	1.15	1.15	1.15	1.15	1.15	1.15
0.13	0.13	0.13	0.13	0.13	0.13	0.13	0.13
0.13	0.13	0.13	0.13	0.13	0.13	0.13	0.13
1.15	1.15	1.15	1.15	1.15	1.15	1.15	1.15
3.19	3.19	3.19	3.19	3.19	3.19	3.19	3.19
6.25	6.25	6.25	6.25	6.25	6.25	6.25	6.25

Increasing Y

Increasing X

Value for Z at (MinX, MinY)

See Also

ddydx, Interpolate, Spline, PFit, Interp3D, InterpMatrix

PrototypeEfficiencyMatrix

Placeholder for non-operative specialized function.

xPrototypeEfficiencyMatrix

Placeholder for non-operative specialized function.

PrototypeSigmaMatrix

Placeholder for non-operative specialized function.

InputByte

Inputs a byte from an I/O port.

Syntax

InputByte(IOAddress)

IOAddress Integer I/O Address 0 - 65535

```
////////////////////////////////////////////////////////////////////////////////
// lInputByte and lOutputByte functions mimic the BASIC and C functions
//
// Note that you really want unsigned char for the return value of lInputByte
//    and ByteToWrite of lOutputByte, and unsigned short for everything
//    else.  However, Excel supports neither unsigned char nor
//    unsigned short, so for simplicity's sake, signed long was used
//    for all of the variables.
//
// lOutputByte returns the tick count (milliseconds Windows has been running).
// This is to enable the calling program to most accurately determine the time
//    the output data was latched (using the system timer).
// Tick count changes every 55 milliseconds (f = 18.1818... Hz).
// Options = 0 => The I/O is performed and the tick count is returned, period.
// Options = 1 => If the tick count taken before outputing the data to the port
//    is different from the tick count taken after outputting data to the port,
//    the function returns 0.  It is up to the calling program to resend the
//    data to the port if such a return time is unacceptable.
// Options = 2 => The routine waits until just after a tick count change and
//    then immediately performs the I/O.  This is probably the most accurate
//    way of getting the system time.  The Options = 1 check is also made in
//    case a long interrupt had taken over.   0 is returned for the same
//    reason as Options = 1.
// Recommend Options = 2 if you want accurate timing, or 0 if you don't care.
////////////////////////////////////////////////////////////////////////////////
signed long _export lInputByte( signed long IOAddress ) ;

signed long _export lOutputByte( signed long IOAddress, signed long ByteToWrite,
        signed long Options ) ;
```

OutputByte

Outputs a byte to an I/O port.

Syntax

OutputByte(IOAddress , ByteToWrite , Options)

IOAddress Integer I/O Address 0 - 65535

```
////////////////////////////////////////////////////////////////////////////////
// lInputByte and lOutputByte functions mimic the BASIC and C functions
//
// Note that you really want unsigned char for the return value of lInputByte
//    and ByteToWrite of lOutputByte, and unsigned short for everything
//    else.  However, Excel supports neither unsigned char nor
//    unsigned short, so for simplicity's sake, signed long was used
//    for all of the variables.
//
// lOutputByte returns the tick count (milliseconds Windows has been running).
// This is to enable the calling program to most accurately determine the time
//    the output data was latched (using the system timer).
// Tick count changes every 55 milliseconds (f = 18.1818... Hz).
// Options = 0 => The I/O is performed and the tick count is returned, period.
// Options = 1 => If the tick count taken before outputing the data to the port
//    is different from the tick count taken after outputting data to the port,
//    the function returns 0.  It is up to the calling program to resend the
//    data to the port if such a return time is unacceptable.
// Options = 2 => The routine waits until just after a tick count change and
//    then immediately performs the I/O.  This is probably the most accurate
//    way of getting the system time.  The Options = 1 check is also made in
//    case a long interrupt had taken over.   0 is returned for the same
//    reason as Options = 1.
// Recommend Options = 2 if you want accurate timing, or 0 if you don't care.
////////////////////////////////////////////////////////////////////////////////
signed long _export lInputByte( signed long IOAddress ) ;
signed long _export lOutputByte( signed long IOAddress, signed long ByteToWrite, signed long Options ) ;
```

VBLookupClosestValue

Exported version of the XlXtrFun worksheet function LookupClosestValue() *g.v.*, for use in other programs such as Visual Basic, C/C++, or FORTRAN.

Declaration in VB module.

```
Public Declare Function VBLookupClosestValue Lib "XLXTRFUN.XLL" ( _
            dArrayOfValues As Double, _
            nNumberOfPoints As Long, _
            dValueToSeek As Double _
            ) As Double
```

Remarks

- Include the declaration at the beginning of any *one* module in each workbook that has modules that use this function. If the keyword `Public` is *not* included, the declaration must be placed at the beginning of every module that uses this function, so it is a good idea to always include `Public`.

Example:

Here is a complete module.

```
Option Explicit

Public Declare Function VBLookupClosestValue Lib "XlXtrFun.xll" _
        (ArrayOfValues As Double, _
        Npts As Long, _
        ValueToSeek As Double) As Double

Sub ExampleOfVBLookupClosestValue()

    Dim l_dArray(5) As Double, _
        l_dValueClosestTo32 As Double

    Dim LF As String * 1
    LF = Chr$(10)

    l_dArray(0) = 10
    l_dArray(1) = 20
    l_dArray(2) = 30
    l_dArray(3) = 40
    l_dArray(4) = 50

    l_dValueClosestTo32 = VBLookupClosestValue(l_dArray(0), 5, 32)

    MsgBox "Array( 0 ) = " & l_dArray(0) & LF & _
            "Array( 1 ) = " & l_dArray(1) & LF & _
            "Array( 2 ) = " & l_dArray(2) & LF & _
            "Array( 3 ) = " & l_dArray(3) & LF & _
            "Array( 4 ) = " & l_dArray(4) & LF & LF & _
            "Value in array closest to 32 is: " & _
                    l_dValueClosestTo32 & LF & LF & _
            "Value in array closest to 35 is: " & _
                    VBLookupClosestValue(l_dArray(0), 5, 35)

End Sub
```

Output:

{bmct bm7.BMP}

- Note that by passing `l_dArray(0)`, you are really passing the reference to the first element of the array.

Function Prototype in C:

```
double __stdcall VBLookupClosestValue(
        double far * Array ,
        unsigned long far * npts ,
        double far * Value ) ;
```

VBIndexOfClosestValue

Exported version of the XlXtrFun worksheet function IndexOfClosestValue() *g.v.*, for use in use in other programs such as Visual Basic, C/C++, or FORTRAN. The first element of the array is at index = 1.

Declaration in VB module.

```
Public Declare Function VBIndexOfClosestValue Lib "XlXtrFun.xll" _
        (ArrayOfValues As Double, _
        Npts As Long, _
        ValueToSeek As Double) As Long
```

Remarks

- Include the declaration at the beginning of any *one* module in each workbook that has modules that use this function. If the keyword `Public` is *not* included, the declaration must be placed at the beginning of every module that uses this function, so it is a good idea to always include `Public`.
- The function will give the 1-based index of the closest value using the same routine as the XlXtrFun worksheet function IndexOfClosestValue().
- Note that the first element of the array you give it is considered index = 1.

Example:

Here is a complete module.

```
Option Explicit

Public Declare Function VBIndexOfClosestValue Lib "XlXtrFun.xll" _
        (ArrayOfValues As Double, _
        Npts As Long, _
        ValueToSeek As Double) As Long

Sub ExampleOfVBIndexOfClosestValue()

    Dim l_dArray(5) As Double, _
        l_dIndexOfValueClosestTo32 As Double

    Dim LF As String * 1
    LF = Chr$(10)

    l_dArray(0) = 10
    l_dArray(1) = 20
    l_dArray(2) = 30
    l_dArray(3) = 40
    l_dArray(4) = 50

    l_dIndexOfValueClosestTo32 = VBIndexOfClosestValue(l_dArray(0), 5, 32)

    MsgBox "Array( 0 ) = " & l_dArray(0) & LF & _
            "Array( 1 ) = " & l_dArray(1) & LF & _
            "Array( 2 ) = " & l_dArray(2) & LF & _
            "Array( 3 ) = " & l_dArray(3) & LF & _
            "Array( 4 ) = " & l_dArray(4) & LF & LF & _
            "Index of value in array closest to 32 is: " & _
                    l_dIndexOfValueClosestTo32 & LF & LF & _
            "Index of value in array closest to 35 is: " & _
                    VBIndexOfClosestValue(l_dArray(0), 5, 35)
End Sub
```

Output:

{bmct bm8.BMP}

- Note that by passing `l_dArray(0)`, you are really passing the address of the first element of the array.

Function Prototype in C:

```
WORD __stdcall VBIndexOfClosestValue(
        double far * Array ,
        unsigned long far * npts ,
        double far * Value ) ;
```

VBLookupClosestValue2D

Exported version of the XlXtrFun worksheet function LookupClosestValue2D() *q.v.*, for use in use in other programs such as Visual Basic, C/C++, or FORTRAN. See the example for the array element numbering.

Declaration in VB module.

```
Public Declare Function VBLookupClosestValue2D Lib "XlXtrFun.xll" _
        (ArrayOfXYValues As Double, _
        ArrayOfXKeys As Double, _
        ArrayOfYKeys As Double, _
        NumberOfXsAndXKeys As Long, _
        NumberOfYsAndYKeys As Long, _
        XValueToSeek As Double, _
        YValueToSeek As Double) As Double
```

Remarks

- Include the declaration at the beginning of any *one* module in each workbook that has modules that use this function. If the keyword `Public` is *not* included, the declaration must be placed at the beginning of every module that uses this function, so it is a good idea to always include `Public`.

Example:

Here is a complete module.

```
Option Explicit

Public Declare Function VBLookupClosestValue2D Lib "XlXtrFun.xll" _
        (ArrayOfXYValues As Double, _
        ArrayOfXKeys As Double, _
        ArrayOfYKeys As Double, _
        NumberOfXsAndXKeys As Long, _
        NumberOfYsAndYKeys As Long, _
        XValueToSeek As Double, _
        YValueToSeek As Double) As Double

Sub ExampleOfVBLookupClosestValue2D()

    Dim l_dArrayOfXYValues(1 To 4, 1 To 5) As Double, _
        l_dArrayOfXKeys(1 To 4) As Double, _
        l_dArrayOfYKeys(1 To 5) As Double, _
        l_dValueClosestTo2_2_4_6 As Double

    Dim l_nNumberOfXsAndXKeys As Long, _
        l_nNumberOfYsAndYKeys As Long

    Dim l_strMessage As String

    Dim LF As String * 1, _
        HT As String * 1
    LF = Chr$(10)
    HT = Chr$(9)

    l_nNumberOfXsAndXKeys = 4
    l_dArrayOfXKeys(1) = 1
        l_dArrayOfXKeys(2) = 2
            l_dArrayOfXKeys(3) = 3
                l_dArrayOfXKeys(4) = 4

    l_nNumberOfYsAndYKeys = 5
    l_dArrayOfYKeys(1) = 1
        l_dArrayOfYKeys(2) = 2
            l_dArrayOfYKeys(3) = 3
```

```
            l_dArrayOfYKeys(4) = 4
                l_dArrayOfYKeys(5) = 5


l_dArrayOfXYValues(1, 1) = 2
    l_dArrayOfXYValues(2, 1) = 5
        l_dArrayOfXYValues(3, 1) = 10
            l_dArrayOfXYValues(4, 1) = 17

l_dArrayOfXYValues(1, 2) = 9
    l_dArrayOfXYValues(2, 2) = 12
        l_dArrayOfXYValues(3, 2) = 17
            l_dArrayOfXYValues(4, 2) = 24

l_dArrayOfXYValues(1, 3) = 28
    l_dArrayOfXYValues(2, 3) = 31
        l_dArrayOfXYValues(3, 3) = 36
            l_dArrayOfXYValues(4, 3) = 43

l_dArrayOfXYValues(1, 4) = 65
    l_dArrayOfXYValues(2, 4) = 68
        l_dArrayOfXYValues(3, 4) = 73
            l_dArrayOfXYValues(4, 4) = 80

l_dArrayOfXYValues(1, 5) = 126
    l_dArrayOfXYValues(2, 5) = 129
        l_dArrayOfXYValues(3, 5) = 134
            l_dArrayOfXYValues(4, 5) = 141

' Build the string containing the array values
l_strMessage = HT & HT & l_dArrayOfXKeys(1) & HT & _
        l_dArrayOfXKeys(2) & HT & _
        l_dArrayOfXKeys(3) & HT & _
        l_dArrayOfXKeys(4) & LF & LF
l_strMessage = l_strMessage & l_dArrayOfYKeys(1) & HT & HT & _
        l_dArrayOfXYValues(1, 1) & HT & _
        l_dArrayOfXYValues(2, 1) & HT & _
        l_dArrayOfXYValues(3, 1) & HT & _
        l_dArrayOfXYValues(4, 1) & LF
l_strMessage = l_strMessage & l_dArrayOfYKeys(2) & HT & HT & _
        l_dArrayOfXYValues(1, 2) & HT & _
        l_dArrayOfXYValues(2, 2) & HT & _
        l_dArrayOfXYValues(3, 2) & HT & _
        l_dArrayOfXYValues(4, 2) & LF
l_strMessage = l_strMessage & l_dArrayOfYKeys(3) & HT & HT & _
        l_dArrayOfXYValues(1, 3) & HT & _
        l_dArrayOfXYValues(2, 3) & HT & _
        l_dArrayOfXYValues(3, 3) & HT & _
        l_dArrayOfXYValues(4, 3) & LF
l_strMessage = l_strMessage & l_dArrayOfYKeys(4) & HT & HT & _
        l_dArrayOfXYValues(1, 4) & HT & _
        l_dArrayOfXYValues(2, 4) & HT & _
        l_dArrayOfXYValues(3, 4) & HT & _
        l_dArrayOfXYValues(4, 4) & LF
l_strMessage = l_strMessage & l_dArrayOfYKeys(5) & HT & HT & _
        l_dArrayOfXYValues(1, 5) & HT & _
        l_dArrayOfXYValues(2, 5) & HT & _
        l_dArrayOfXYValues(3, 5) & HT & _
        l_dArrayOfXYValues(4, 5) & LF & LF

l_dValueClosestTo2_2_4_6 = VBLookupClosestValue2D(l_dArrayOfXYValues(1, 1), _
        l_dArrayOfXKeys(1), l_dArrayOfYKeys(1), _
        l_nNumberOfXsAndXKeys, l_nNumberOfYsAndYKeys, _
```

```
            2.2, 4.6)

    MsgBox l_strMessage & _
            "Value in array closest to ( 2.2 , 4.6 ) is: " & _
                    l_dValueClosestTo2_2_4_6 & LF & LF & _
            "Value in array closest to ( 2.6 , 4.6 ) is: " & _
                    VBLookupClosestValue2D(l_dArrayOfXYValues(1, 1), _
                            l_dArrayOfXKeys(1), l_dArrayOfYKeys(1), _
                            l_nNumberOfXsAndXKeys, l_nNumberOfYsAndYKeys, _
                            2.6, 4.6), _
            , "Example Of VBLookupClosestValue2D"

End Sub
```

Output:

{bmct bm9.BMP}

- Note that by passing `l_dArrayOfXYValues(0, 0)` and `l_dArrayOfXKeys(0)`, you are really passing the address of the first element of the arrays.

Function Prototype in C:

```
double __stdcall VBLookupClosestValue2D(
        double far * XYArray ,
        double far * ArrayOfXKeys ,
        double far * ArrayOfYKeys ,
        unsigned long far * NumberOfXsAndXKeys ,
        unsigned long far * NumberOfYsAndYKeys ,
        double far * XValue ,
        double far * YValue ) ;
```

VBXatY

Exported version of the XlXtrFun worksheet function XatY() *q.v.*

Declaration in VB module.

```
Public Declare Function VBXatY Lib "XlXtrFun.xll" _
        (Xs As Double, _
        Ys As Double, _
        Npts As Long, _
        SolveForType As Integer, _
        GuessXAndReturnX As Double, _
        YtoSeek As Double) As Long
```

Remarks

- Include the declaration at the beginning of any *one* module in each workbook that has modules that use this function. If the keyword `Public` is not included, the declaration must be placed at the beginning of every module that uses this function, so it is a good idea to always include `Public`.

Example:

Here is a complete module.

```
Option Explicit

Public Declare Function VBXatY Lib "XlXtrFun.xll" ( _
            dGivenXs As Double, _
            dGivenYs As Double, _
            nNumberOfPoints As Long, _
            nSolveForType As Integer, _
            dGuessX As Double, _
            dYtoSeek As Double _
            ) As Long

'*****************************************************************************
'*****************************************************************************
Sub TestVBXAtY()

    Dim l_nFunctionReturnValue As Integer
    Dim l_strResultString As String
    Dim l_dFindXAtThisY As Double, _
        l_dGuessXatY As Double, _
        l_dGuessXatPeak As Double, _
        l_dGuessXatValley As Double, _
        l_dGuessAndReturnedXatPeak As Double, _
        l_dGuessAndReturnedXatValley As Double, _
        l_dGuessAndReturnedXatY As Double

    GetInputDataFromSheet1
    l_dFindXAtThisY = Sheets("Sheet1").Range("Sheet1FindXAtThisY")
    l_dGuessXatY = Sheets("Sheet1").Range("Sheet1GuessX")

    l_dGuessXatPeak = dX1s(VBIndexOfClosestValue(dY1s(0), nNumberOf1s,
Application.Max(dY1s)))
    l_dGuessAndReturnedXatPeak = l_dGuessXatPeak

    l_dGuessXatValley = dX1s(VBIndexOfClosestValue(dY1s(0), nNumberOf1s,
Application.Min(dY1s)))
    l_dGuessAndReturnedXatValley = l_dGuessXatValley

    l_dGuessAndReturnedXatY = l_dGuessXatY

    l_nFunctionReturnValue = VBXatY( _
```

```
            dX1s(0), _
            dY1s(0), _
            nNumberOf1s, _
            CInt(Asc("P")), _
            l_dGuessAndReturnedXatPeak, l_dFindXAtThisY)

    l_nFunctionReturnValue = VBXatY( _
            dX1s(0), _
            dY1s(0), _
            nNumberOf1s, _
            CInt(Asc("V")), _
            l_dGuessAndReturnedXatValley, l_dFindXAtThisY)

    l_nFunctionReturnValue = VBXatY( _
            dX1s(0), _
            dY1s(0), _
            nNumberOf1s, _
            CInt(Asc("Y")), _
            l_dGuessAndReturnedXatY, l_dFindXAtThisY)

    l_strResultString = l_strResultString & Chr$(10) & "VBXatY found Peak Y at X = " & _
            l_dGuessAndReturnedXatPeak & " starting from X = " & l_dGuessXatPeak

    l_strResultString = l_strResultString & Chr$(10) & "VBXatY found Valley Y at X = " &
_
            l_dGuessAndReturnedXatValley & " starting from X = " & l_dGuessXatValley

    l_strResultString = l_strResultString & Chr$(10) & "VBXatY found Y = " &
l_dFindXAtThisY & " at X = " & l_dGuessAndReturnedXatY & _
            " starting from X = " & l_dGuessXatY

    MsgBox l_strResultString

End Sub
```

Function Prototype in C:

```
unsigned long __stdcall VBXatY( double far * Xs , double far * Ys ,
    unsigned long far * npts , short far * SolveForType ,
    double far * GuessX , double far * YtoSeek ) ;
```

Returns 43 if extrapolation was required,

Notes on Error Messages

#VALUE!

This error is returned by Excel when it cannot convert the arguments you have given a particular function to the arguments XlXtrFun requires. For example, if a function requires a number, and you give it a text value. Excel cannot convert the text to a number, so it returns the #VALUE! error. See Excel #VALUE! help for an excellent summary of the causes of that error.

#NUM!

This error is usually returned by XlXtrFun when it cannot make sense out of what you gave it.

See also

Excel #VALUE! help, Excel #NUM! help

#define _VBInterpolate 106
#define _VBXatY 108
#define _VBdydx 110
#define _VBddydx 111
#define _VBInterp3D 112
#define _VBInterpMatrix 113

[$][#][K]q.v.: Abbreviation for the Latin *quid vide*, "which see".

[$] q.v. Definition
[#] 31H._P_
[K] q v;q. v.;q.v.;qv